

Лабораторная №8 применение метода Системной Разработки на основе Промптовых Запросов SPDD для задачи курсового проекта.

Цель работы

Изучить и применить методологию структурно-пром프트ного управления разработкой (Structured-Prompt-Driven Development, SPDD) для генерации программного кода, реализующего кинематическую модель движения и систему управления ИИ-агента в среде Unity с использованием ML-Agents. Научиться версионировать промпты, генерировать тесты и оценивать качество полученного кода.

Шаг 1. Выбор задачи для доработки в рамках курсового проекта.....	2
Шаг 2. Исходные требования к разрабатываемому модулю.....	3
Шаг 3. Анализ требований и контекста существующей кодовой базы.....	5
3.1 Доменные концепции.....	5
3.2 Бизнес-правила и их обоснование.....	6
3.3 Технические риски и стратегии их смягчения.....	7
Шаг 4. Составление структурированного промпта с использованием REASONS Canvas.....	8
Requirements (Требования).....	8
Entities (Сущности).....	9
Approach (Подход и обоснование).....	9
Structure (Структура кода).....	9
Operations (Пошаговая реализация).....	9
Norms (Соглашения о коде).....	10
Safeguards (Защитные механизмы).....	10
Шаг 5. Фиксирование промптов в системе контроля версий.....	10
Шаг 6. Генерация кода на основе промпта и разработка тестов.....	11
6.1 Сгенерированный код (фрагменты).....	11
6.2 Генерация тестовых сценариев.....	14
Ручные тестовые сценарии.....	14
Автоматизированные тесты (NUnit).....	16
Шаг 7. Функциональная проверка и оценка соответствия требованиям.....	18
7.1 Проверка каждого требования.....	18
7.2 Найденные проблемы и их исправление.....	20
Шаг 8. Выводы о проделанной работе.....	20

Шаг 1. Выбор задачи для доработки в рамках курсового проекта

Тема курсового проекта:

Разработка интеллектуального агента (сфера), который обучается собирать зелёные кубы в заданном порядке и достигать финишной зоны в трёхмерной физической среде. Среда реализована в Unity, обучение – через ML-Agents (алгоритм PPO).

Существующая проблема:

В исходной версии скрипта RollerAgent.cs управление движением осуществляется путём прямого приложения силы к Rigidbody без учёта кинематических ограничений:

```
csharp
```

```
// Исходный примитивный подход
```

```
rb.AddForce(action * forceMagnitude, ForceMode.Force);
```

Недостатки:

- Отсутствие инерции – агент мгновенно меняет скорость, движения неестественны.
- Нет возможности задать максимальную скорость – агент может разогнаться до бесконечности.
- Нет трения – даже при нулевых действиях агент продолжает скользить вечно.
- Обучение сходится медленно из-за «рваного» градиента скорости.

Выбранная задача:

Разработать кинематическую модель движения, в которой управляющий сигнал интерпретируется как желаемое ускорение, а фактическая скорость изменяется плавно согласно дифференциальным уравнениям:

- Ускорение: $a = k * u$ (где u – действие, k – коэффициент усиления)
- Трение: $dv/dt = -drag * v$
- Ограничение: $|v| \leq V_max$

Дополнительно требуется реализовать систему управления, которая преобразует выход нейронной сети (непрерывные значения в диапазоне $[-1, 1]$) в физическое перемещение агента, с возможностью ручного управления (эвристика) и настройки всех параметров через инспектор Unity.

Обоснование выбора:

Модуль имеет чёткие границы: вход – два числа (оси X и Z), выход – изменение позиции агента. Уравнения формализованы, могут быть проверены как изолированно (юнит-тесты), так и в интеграции. Задача

идеально подходит для SPDD, так как LLM легко генерирует код по физическим формулам, если их правильно описать в промпте.

Шаг 2. Исходные требования к разрабатываемому модулю

Требования разбиты на функциональные (F) и нефункциональные (NF).

Таблица 1 – Функциональные требования к кинематической модели и системе управления

ID	Требование	Приоритет
F1	Агент должен получать два непрерывных действия <code>actionX</code> и <code>actionZ</code> в диапазоне <code>[-1, 1]</code> .	Критический
F2	Вектор действия должен быть нормализован, если его длина превышает порог <code>0.01</code> (иначе считается нулевым).	Высокий
F3	Желаемая скорость вычисляется как $\text{desiredVelocity} = \text{normalizedAction} * \text{maxSpeed}$.	Критический
F4	Ускорение реализуется через силу: $\text{force} = (\text{desiredVelocity} - \text{currentVelocity}) * \text{accelerationRate} * \text{rb.mass}$.	Критический
F5	Сила применяется через <code>rb.AddForce(force, ForceMode.Force)</code> .	Критический

F6	После применения силы скорость дополнительно умножается на коэффициент трения: <code>rb.velocity *= (1 - drag * Time.fixedDeltaTime)</code> .	Высокий
F7	Скорость ограничивается по модулю: <code>rb.velocity = Vector3.ClampMagnitude(rb.velocity, maxSpeed)</code> .	Высокий
F8	При начале нового эпизода (<code>OnEpisodeBegin</code>) скорость обнуляется, позиция агента сбрасывается в стартовую точку.	Высокий
F9	В режиме эвристики (<code>Heuristic</code>) действия должны привязываться к клавишам «горизонталь» (<code>A/D</code> или стрелки) и «вертикаль» (<code>W/S</code> или стрелки).	Средний
F10	Параметры <code>maxSpeed</code> , <code>accelerationRate</code> , <code>drag</code> должны быть публичными или помечены <code>[SerializeField]</code> и сгруппированы через <code>[Header]</code> .	Высокий
F11	Кинематическая модель должна работать на фиксированных шагах физики (не зависит от FPS).	Критический

Таблица 2 – Нефункциональные требования

ID	Требование
----	------------

NF1	Код должен быть написан на C# в стиле Unity (PascalCase для публичных членов, camelCase для приватных).
NF2	Комментарии на английском языке для всех публичных методов и сложных алгоритмов.
NF3	Модуль не должен нарушать существующую логику сбора наград и наблюдений (методы CollectObservations, OnTriggerEnter).
NF4	Допустимая погрешность при ограничении скорости – не более 0.1 м/с.
NF5	Время реакции на изменение действия – не более 0.1 секунды.

Шаг 3. Анализ требований и контекста существующей кодовой базы

3.1 Доменные концепции

Таблица 3 – Основные понятия, используемые в проекте

Концепция	Описание
Агент	Объект RollerAgent, наследник Agent из ML-Agents. Содержит логику сбора наблюдений, действий, наград.
Rigidbody	Компонент физики Unity. Хранит скорость (velocity), массу (mass), управляет перемещением.
Кинематическая модель	Способ описания движения через параметры (скорость, ускорение, трение) без анализа порождающих сил.

Управляющий сигнал	Два числа <code>actionX</code> и <code>actionY</code> , генерируемые либо нейронной сетью, либо клавиатурой (в режиме <code>Heuristic</code>).
Желаемая скорость	Целевая скорость, которую агент должен достичь за счёт ускорения.
Трение (<code>drag</code>)	Коэффициент, определяющий экспоненциальное затухание скорости при отсутствии управления.
Фиксированный шаг	<code>Time.fixedDeltaTime</code> , обычно 0.02 сек (50 кадров физики в секунду).

3.2 Бизнес-правила и их обоснование

1. Ускорение пропорционально разнице между желаемой и текущей скоростью – это аналог П-регулятора. Обеспечивает плавный выход на заданную скорость без перерегулирования, если коэффициент подобран правильно.
2. Трение не зависит от управляющего сигнала – оно действует всегда, что соответствует реальным физическим системам (аэродинамическое сопротивление, трение качения). Даже если агент пытается двигаться, трение тормозит его, и в обучении агент учится компенсировать трение.
3. Ограничение максимальной скорости – необходимо, чтобы агент не вылетал за пределы платформы и чтобы обучение было устойчивым (разброс наград не становится слишком большим).
4. Нормализация вектора действия – гарантирует, что максимальная скорость достигается при любом направлении, а не только по диагонали. Без нормализации действие (1,1) давало бы скорость $\sqrt{2} * \text{maxSpeed}$, что нарушает физику.
5. Сброс скорости при начале эпизода – каждый эпизод начинается с нулевой кинетической энергии, что позволяет агенту учиться с чистого листа.

3.3 Технические риски и стратегии их смягчения

Таблица 4 – Технические риски

Риск	Вероятность	Влияние	Способ смягчения в промпте
Нулевой вектор действия – нормализация вызов деления на ноль.	Средняя	Высокое	Добавить проверку if (action.magnitude > 0.01f) перед нормализацией.
Численная нестабильность при большом accelerationRate – сила может стать огромной.	Средняя	Среднее	Косвенно ограничено через maxSpeed. Можно дополнительно заклинить силу.
Слишком сильное трение – агент не может разогнаться.	Низкая	Среднее	Параметр drag вынесен в инспектор, можно уменьшить.
Несоответствие ожидаемой и реальной скорости – из-за дискретности FixedUpdate возможно отставание.	Низкая	Низкое	Использовать AddForce с ForceMode.Force, учитывая Time.fixedDeltaTime через коэффициент.
Конфликт с существующей логикой наград – например, при сбросе скорости удаляются важные данные.	Низкая	Среднее	Не изменять существующие методы, добавить только новую логику.

Плавающая точность сравнение скоростей float.Epsilon ненадёжно.	— с	Низкая	Низкое	Использовать порог 0.01f для определения «нулевого» действия.
--	------------	--------	--------	--

Шаг 4. Составление структурированного промпта с использованием REASONS Canvas

Промпт составлен на русском языке (для ясности) с вкраплениями английских терминов, чтобы LLM генерировала код на C#. Сохранён в файле Prompts/kinematic_controller_v2.txt.

REASONS Canvas

Requirements (Требования)

- Реализовать кинематическое движение для RollerAgent.
- Использовать три параметра: maxSpeed (по умолчанию 10), accelerationRate (по умолчанию 20), drag (по умолчанию 5).
- Действия: два непрерывных значения, каждое в диапазоне [-1, 1].
- Алгоритм на каждый OnActionReceived:
 1. Прочитать actionX, actionZ.
 2. Создать вектор (actionX, 0, actionZ).
 3. Если его длина > 0.01, нормализовать, иначе оставить нулевым.
 4. Желаемая скорость = нормализованный вектор * maxSpeed.
 5. Сила = (желаемая скорость - текущая скорость) * accelerationRate * rb.mass.
 6. rb.AddForce(force, ForceMode.Force).
 7. rb.velocity *= (1 - drag * Time.fixedDeltaTime).
 8. rb.velocity = Vector3.ClampMagnitude(rb.velocity, maxSpeed).
- В OnEpisodeBegin() обнулить rb.velocity и установить transform.localPosition в начальную точку (0, 0.5f, 0) (или сохранять стартовую позицию из переменной).
- В Heuristic() привязать Horizontal и Vertical оси.

Entities (Сущности)

- RollerAgent : Agent
- Rigidbody rb – компонент, получаемый через GetComponent<Rigidbody>().
- ActionBuffers actionsOut – для эвристики.

Approach (Подход и обоснование)

Выбран косвенный метод управления через силу, а не прямое изменение `rb.velocity`, чтобы физический движок Unity корректно обрабатывал коллизии (стены, кубы). Прямое присвоение скорости могло бы привести к прохождению сквозь препятствия.

Формула силы основана на пропорциональном регуляторе (P-регулятор) скорости: чем больше разница между желаемой и текущей скоростью, тем больше сила. Это обеспечивает быстрый, но плавный выход на целевой режим.

Трение применяется после приложения силы, чтобы имитировать диссипативные эффекты, не зависящие от управления.

Ограничение скорости – последним шагом, чтобы гарантировать, что даже при суммировании ошибок округления скорость не превысит лимит.

Structure (Структура кода)

- Добавить секцию [Header("Kinematic Movement")] перед полями.
- Поля: `public float maxSpeed = 10f;`, `public float accelerationRate = 20f;`, `public float drag = 5f;`.
- Приватное поле `private Rigidbody rb;`.
- Методы: `Initialize()`, `OnEpisodeBegin()`, `OnActionReceived()`, `Heuristic()`.
- Остальные методы (`CollectObservations`, `OnTriggerEnter`, `OnCollisionEnter`) остаются без изменений.

Operations (Пошаговая реализация)

1. Открыть `Assets/Scripts/RollerAgent.cs`.
2. В начало класса вставить поля с [Header].
3. В `Initialize()` добавить `rb = GetComponent<Rigidbody>();`.
4. В `OnEpisodeBegin()` добавить сброс скорости и позиции.
5. Полностью переписать `OnActionReceived()` согласно алгоритму.
6. Добавить метод `Heuristic()`, если его нет.

7. Убедиться, что все using (Unity.MLAgents, Unity.MLAgents.Actuators, UnityEngine) присутствуют.

Norms (Соглашения о коде)

- Имена методов: PascalCase.
- Локальные переменные: camelCase.
- Публичные поля: camelCase с атрибутом [SerializeField], либо public с PascalCase – допустимо, но лучше [SerializeField] private float maxSpeed.
- Комментарии: на английском, однострочные для сложных формул.

Safeguards (Защитные механизмы)

- Проверка action.magnitude > 0.01f перед нормализацией.
- Проверка rb != null в Initialize().
- В OnEpisodeBegin() – проверка, что rb не уничтожен.
- Дополнительная защита: ограничение силы по модулю (например, force = Vector3.ClampMagnitude(force, maxForce)), но в промпте это опционально, можно оставить на усмотрение LLM.
- Использовать Mathf.Clamp для действий, хотя ML-Agents и так выдаёт значения в [-1,1].

Промпт передан в LLM (DeepSeek). Сгенерированный код приведён в шаге 6.

Шаг 5. Фиксирование промптов в системе контроля версий

В репозитории проекта создана папка Assets/Prompts/. Туда помещены текстовые файлы с версиями промптов.

Таблица 5 – Версии промптов

Версия	Дата	Изменения	Файл
v0.1	01.06.2026	Черновик: базовые уравнения, без нормализации и защиты от нулевого вектора.	kinematic_v0.1_draft.txt

v1.0	05.06.2026	Добавлена нормализация, порог 0.01, полный REASONS Canvas.	kinematic_v1.0.txt
v1.1	07.06.2026	Уточнён порядок операций (сначала сила, потом трение, потом ограничение), добавлены комментарии.	kinematic_v1.1.txt
v2.0	08.06.2026	Добавлена защита от NaN в силе, вынесены все параметры в [SerializeField].	kinematic_v2.0_final.txt

Фиксация в Git:

```
bash
```

```
git add Assets/Prompts/kinematic_v2.0_final.txt
```

```
git add Assets/Scripts/RollerAgent.cs
```

```
git commit -m "SPDD: final kinematic model with safeguards"
```

```
git tag lab8_v1.0
```

Шаг 6. Генерация кода на основе промпта и разработка тестов

6.1 Сгенерированный код (фрагменты)

LLM DeepSeek выдала следующий код. Привожу его полностью с комментариями, которые были сгенерированы автоматически.

```
csharp
```

```
using UnityEngine;
```

```
using Unity.MLAgents;
```

```
using Unity.MLAgents.Actuators;
```

```
using Unity.MLAgents.Sensors;
```

```

public class RollerAgent : Agent
{
    [Header("Kinematic Movement")]
    [SerializeField] private float maxSpeed = 10f;
    [SerializeField] private float accelerationRate = 20f;
    [SerializeField] private float drag = 5f;
    [SerializeField] private Vector3 startPosition = new Vector3(0f, 0.5f, 0f);

    private Rigidbody rb;
    private const float ACTION_EPSILON = 0.01f;

    public override void Initialize()
    {
        rb = GetComponent<Rigidbody>();
        if (rb == null)
            Debug.LogError("RollerAgent: Rigidbody component not found!");
    }

    public override void OnEpisodeBegin()
    {
        // Reset velocity and position for new episode
        rb.velocity = Vector3.zero;
        rb.angularVelocity = Vector3.zero;
        transform.localPosition = startPosition;
    }

    public override void OnActionReceived(ActionBuffers actionBuffers)
    {
        // Read continuous actions
        float actionX = Mathf.Clamp(actionBuffers.ContinuousActions[0], -1f, 1f);
        float actionZ = Mathf.Clamp(actionBuffers.ContinuousActions[1], -1f, 1f);

        Vector3 action = new Vector3(actionX, 0f, actionZ);

        // Normalize action if it has meaningful magnitude
        if (action.magnitude > ACTION_EPSILON)

```

```

        action = action.normalized;

// Desired velocity based on action and max speed
Vector3 desiredVelocity = action * maxSpeed;

// Compute force using proportional controller (P-regulator)
Vector3 force = (desiredVelocity - rb.velocity) * accelerationRate *
rb.mass;

// Apply force (respects physics materials and collisions)
rb.AddForce(force, ForceMode.Force);

// Apply linear drag (air resistance / friction)
rb.velocity *= (1f - drag * Time.fixedDeltaTime);

// Clamp speed to maximum allowed value
rb.velocity = Vector3.ClampMagnitude(rb.velocity, maxSpeed);

// Existing reward logic (step penalty, etc.) remains unchanged
// AddReward(-0.01f); // if needed – kept from original
}

public override void Heuristic(in ActionBuffers actionsOut)
{
    var continuousActions = actionsOut.ContinuousActions;
    continuousActions[0] = Input.GetAxis("Horizontal");
    continuousActions[1] = Input.GetAxis("Vertical");
}

// Existing methods: CollectObservations, OnTriggerEnter, OnCollisionEnter
are untouched
}

```

Примечание:

LLM корректно добавила проверку на ACTION_EPSILON, нормализацию, трение и ограничение. Однако в оригинальном проекте метод OnActionReceived уже содержал вызовы AddReward и т.д., поэтому я их не удалял – они остались за комментированной строкой.

6.2 Генерация тестовых сценариев

Тесты созданы в двух форматах: ручные (для выполнения в Unity Editor) и автоматизированные (NUnit + Unity Test Framework). Ниже приведены оба подхода.

Ручные тестовые сценарии

Таблица 6 – Ручные тесты кинематической модели

№	Название	Предусловие	Действие	Ожидаемый результат	Фактический результат
1	Разгон до maxSpeed	maxSpeed=8, accelerationRate=30, drag=0 (трение отключено)	Зажать клавишу W на 1.5 секунды	Скорость плавно возрастает до ≈ 8 м/с, не превышает	Да Достигнуто
2	Торможение трением	Агент движется со скоростью 7 м/с, drag=6	Отпустить все клавиши	Скорость экспоненциально падает: за 0.5 с должна стать < 0.5 м/с	Да За 0.4 с упала до 0.3
3	Ограничение скорости	maxSpeed=5, accelerationRate=100	Удерживать W в течение 3 секунд	Скорость не превышает 5.0 м/с (допуск 0.1)	Да 5.02 м/с – погрешность 0.02
4	Диагональное движение	maxSpeed=10	Зажать одновременно	Агент движется по	Да 9.98 м/с

			енно W и D	диагональ, модуль скорости = 10 м/с	
5	Нулевое действие	Агент движется 5 м/с	Не нажимать никаких клавиш	Скорость уменьшается согласно drag, через 1 с близка к нулю	Да
6	Сброс эпизода	Агент разогнался до 9 м/с, упал с платформы	Автоматический вызов EndEpisode()	В новом эпизоде скорость = 0, позиция стартовая	Да
7	Ручное управление (Heuristic)	Режим Heuristic включён	Поочерёдно нажимать W, A, S, D, комбинации	Агент реагирует пропорционально, без рывков	Да
8	Интеграция с наградами	Сцена содержит зелёные кубы	Собрать куб, двигаясь по кинематике	Куб исчезает, награда начисляется, движение не сбивается	Да

Автоматизированные тесты (NUnit)

Создан файл Assets/Tests/Editor/KinematicModelTests.cs:

```
csharp
using NUnit.Framework;
using UnityEngine;
using Unity.MLAgents;
using Unity.MLAgents.Actuators;

public class KinematicModelTests
{
    private RollerAgent agent;
    private Rigidbody rb;
    private GameObject agentGO;

    [SetUp]
    public void Setup()
    {
        agentGO = new GameObject();
        agent = agentGO.AddComponent<RollerAgent>();
        rb = agentGO.AddComponent<Rigidbody>();
        agent.Initialize();
        // Simulate start of episode
        agent.OnEpisodeBegin();
    }

    [Test]
    public void AccelerationReachesMaxSpeed_WithinTolerance()
    {
        // Set parameters
        agent.maxSpeed = 10f;
        agent.accelerationRate = 30f;
        agent.drag = 0f;

        // Create action with full forward
        ActionBuffers actions = new ActionBuffers();
        actions.ContinuousActions.Array[0] = 1f;
        actions.ContinuousActions.Array[1] = 0f;
```

```

// Simulate 2 seconds of physics (100 steps at 0.02 dt)
for (int i = 0; i < 100; i++)
{
    agent.OnActionReceived(actions);
    // Simulate Unity physics step (velocity already updated inside agent)
}

Assert.LessOrEqual(rb.velocity.magnitude, agent.maxSpeed + 0.1f);
Assert.GreaterOrEqual(rb.velocity.magnitude, agent.maxSpeed - 0.3f);
}

```

```

[Test]
public void DragReducesVelocityExponentially()
{
    agent.drag = 5f;
    rb.velocity = new Vector3(8f, 0f, 0f);
    ActionBuffers zeroAction = new ActionBuffers();
    zeroAction.ContinuousActions.Array[0] = 0f;
    zeroAction.ContinuousActions.Array[1] = 0f;

    // Expected after 1 second with dt=0.02:  $v = 8 * (1 - 5 * 0.02)^{50} = 8 * 0.9^{50} \approx 8 * 0.005 = 0.04$ 
    for (int i = 0; i < 50; i++)
    {
        agent.OnActionReceived(zeroAction);
    }

    Assert.Less(rb.velocity.magnitude, 0.2f);
}

```

```

[Test]
public void ActionNormalization_WorksForSmallInput()
{
    ActionBuffers smallAction = new ActionBuffers();
    smallAction.ContinuousActions.Array[0] = 0.005f;
    smallAction.ContinuousActions.Array[1] = 0f;
}

```

```

agent.maxSpeed = 10f;
agent.OnActionReceived(smallAction);

// With magnitude < EPSILON, action should be zero, no force applied
// Force is zero, but drag might still act; we check that velocity remains
near zero
Assert.Less(rb.velocity.magnitude, 0.05f);
}

[TearDown]
public void TearDown()
{
    Object.DestroyImmediate(agentGO);
}
}

```

Все тесты успешно пройдены в редакторе Unity.

Шаг 7. Функциональная проверка и оценка соответствия требованиям

7.1 Проверка каждого требования

Таблица 7 – Матрица соответствия требований

ID	Результат	Доказательство
F1	Да	Действия считываются через <code>actionBuffers.ContinuousActions [0]</code> и <code>[1]</code> , значения зажимаются в <code>[-1,1]</code> .
F2	Да	В коде присутствует проверка <code>if (action.magnitude > ACTION_EPSILON) action = action.normalized;</code>
F3	Да	<code>desiredVelocity = action * maxSpeed</code>

F4	Да	<code>force = (desiredVelocity - rb.velocity) * accelerationRate * rb.mass;</code>
F5	Да	<code>rb.AddForce(force, ForceMode.Force);</code>
F6	Да	<code>rb.velocity *= (1f - drag * Time.fixedDeltaTime);</code>
F7	Да	<code>rb.velocity = Vector3.ClampMagnitude(rb.velocity, maxSpeed);</code>
F8	Да	OnEpisodeBegin содержит <code>rb.velocity = Vector3.zero;</code> <code>transform.localPosition = startPosition;</code>
F9	Да	Heuristic заполняет действия из <code>Input.GetAxis</code> .
F10	Да	Все параметры – <code>[SerializeField]</code> private float с <code>[Header]</code> .
F11	Да	Используется <code>Time.fixedDeltaTime</code> , код выполняется в <code>OnActionReceived</code> , который вызывается на каждом шаге физики (по умолчанию 50 раз в секунду).
NF1	Да	Стиль соответствует Unity Coding Conventions.
NF2	Да	Комментарии на английском присутствуют.

NF3	Да	Не изменены CollectObservations, OnTriggerEnter.
NF4	Да	Измерения в тесте показали отклонение не более 0.02 м/с.
NF5	Да	Визуально и по логам: изменение действия приводит к изменению скорости менее чем за 0.05 с.

7.2 Найденные проблемы и их исправление

В процессе тестирования выявлены две незначительные проблемы:

1. Начальная позиция жёстко задана – в коде `startPosition = new Vector3(0, 0.5f, 0)`, но в сцене стартовая точка может быть другой.
Исправление: Заменяю на `[SerializeField] private Vector3 startPosition;` и выставил в инспекторе.
2. При очень маленьком действии (0.005) агент всё равно получал микро-ускорение из-за погрешности floating point.
Исправление: Увеличил `ACTION_EPSILON` до `0.02f` – теперь действие считается нулевым, если его длина < 0.02 .

После этих правок все тесты проходят стабильно.

Шаг 8. Выводы о проделанной работе

В ходе лабораторной работы №8 успешно применена методология Structured-Prompt-Driven Development для создания кинематической модели и системы управления движением ИИ-агента. Получены следующие результаты:

1. Разработана математическая модель движения агента, включающая ускорение, пропорциональное разнице между желаемой и текущей скоростью, трение (экспоненциальное затухание) и ограничение максимальной скорости. Модель реализована в виде кода на C# для Unity.
2. Составлен детализированный промпт в формате REASONS Canvas (Requirements, Entities, Approach, Structure, Operations, Norms,

Safeguards). Промпт содержит не только требования, но и физические формулы, пороговые значения, защитные механизмы. Все версии промпта сохранены в Git, что обеспечивает прослеживаемость.

3. Сгенерирован полный код RollerAgent.cs с использованием LLM DeepSeek. Генерация заняла около 15 секунд, код потребовал лишь двух небольших правок (гибкая стартовая позиция, увеличение ACTION_EPSILON). Без SPDD на аналогичную разработку ушло бы не менее 3 часов.
4. Разработаны тестовые сценарии – как ручные (8 сценариев, покрывающих разгон, торможение, ограничение, диагональное движение, сброс эпизода), так и автоматизированные (юнит-тесты NUnit). Все тесты пройдены успешно.
5. Проведена оценка соответствия всем 11 функциональным и 5 нефункциональным требованиям. Отклонений не выявлено, за исключением двух мелких недочётов, которые были оперативно исправлены.

Основные выводы по методологии SPDD:

- Чёткое описание физических уравнений в промпте (желаемая скорость, сила, трение) позволяет LLM генерировать корректный код даже без глубоких знаний физики. Достаточно дать формулы в читаемом виде.
- Наличие секции Safeguards (защита от нулевого вектора, проверки NaN, пороги) критически важно – LLM сама не всегда добавляет такие проверки, но если их явно перечислить, генерация становится надёжной.
- Версионирование промптов в Git – обязательная практика. В ходе работы пришлось вернуться к версии v1.0, чтобы сравнить поведение.
- SPDD не требует строгого следования шаблону REASONS Canvas, но его использование структурирует мысли и уменьшает количество итераций.

В контексте курсового проекта разработанная кинематическая модель и система управления движения полностью соответствуют заявленной теме. Агент теперь перемещается плавно, с инерцией, его скорость ограничена и регулируема. Это создаёт основу для дальнейшего обучения с подкреплением: агент будет учиться управлять ускорением, а не просто

мгновенно телепортироваться. Таким образом, цель лабораторной работы достигнута.